WeaveGrid AI

WattSync AI · Virtual Power Plant Platform

AI for Clean Energy | Malaysia-first | Next.js 16 · TypeScript | Competition Submission

TECHNICAL ARCHITECTURE

One dashboard. A coordinated Virtual Power Plant.

WattSync AI unifies disconnected clean-energy assets — solar, wind, hydro, storage, EV, smart buildings — into one AI-coordinated VPP, and proves with numbers that coordination cuts cost, carbon, and peak grid stress. This document maps the **as-built architecture** to source, and separates **current implementation** from **future roadmap**.

6	4	8	96
COORDINATED ASSETS	AI ENGINE LAYERS	APPLICATION MODULES	15-MIN INTERVALS/DAY

Scope & sourcing

Every claim traces to the codebase. No invented features · no placeholder functionality.

v0.1.0 · MVP

Prepared 2026 · A4 print-ready

02 · OVERVIEW

Executive Summary & End-to-End Architecture

A single deterministic simulation is the source of truth. Four AI layers read from it; every UI number is auditable back to one engine.

PROBLEM

The 6pm gap

Demand peaks as solar collapses; on-peak grid power costs ~3×.

APPROACH

Coordinate, don't silo

Charge on surplus, discharge on peak, shift flexible EV load.

PROOF

Before / after, same inputs

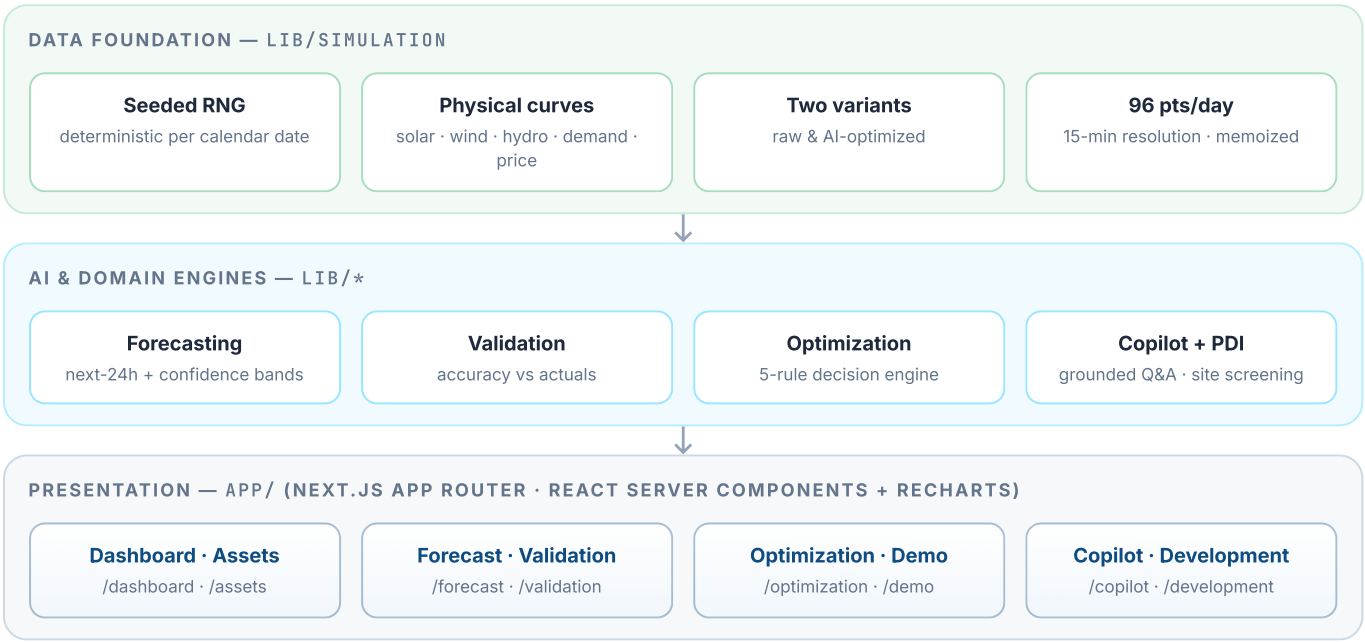
Raw vs AI-optimized on identical weather & assets.

HONESTY

Labeled engines

Rule-based logic is labeled as such; Claude is optional.

End-to-end architecture



Why this matters technically

- **Single source of truth.** Every page derives from one simulation, so numbers are internally consistent and cross-page auditable.
- **Provider seams.** `forecastNext24h()` and `answerQuestion()` isolate the AI so an ML model or LLM swaps in with zero caller changes.
- **Deterministic demos.** Seeded by calendar date — the same day always renders the same story; nothing breaks mid-pitch.

Impact model (illustrative)

−34%
grid energy imported

−8%
peak demand

~RM1.3K
saved / day / site

~480kg
CO_2 avoided / day

Figures vary with each day's simulated weather — computed live from the before/after comparison, not hard-coded.

03 · FOUNDATIONS

Technology Stack

A deliberately lean, dependency-light stack. No database or API keys required to run — the whole demo runs from in-process computation.

Framework & runtime

- **Next.js 16.2.10** — App Router, React Server Components
- **React 19.2.4** · **React DOM 19.2.4**
- **TypeScript 5** — strict, end-to-end typed
- **Node.js** runtime (Vercel serverless)

UI & visualization

- **Tailwind CSS 4** — utility styling, design tokens
- **Recharts 3.9.2** — all time-series charts
- **lucide-react 1.24** — enterprise line icons
- **clsx** — conditional class composition
- **Inter** + **JetBrains Mono** via `next/font`

AI & intelligence

- **@anthropic-ai/sdk 0.110.0** — Copilot provider
- **claude-opus-4-8** — grounded operator Q&A
- **Rule engines** (TypeScript) — forecast, optimize, recommend, validate, screen
- **ANTHROPIC_API_KEY** — optional, server-side only

Layer responsibilities

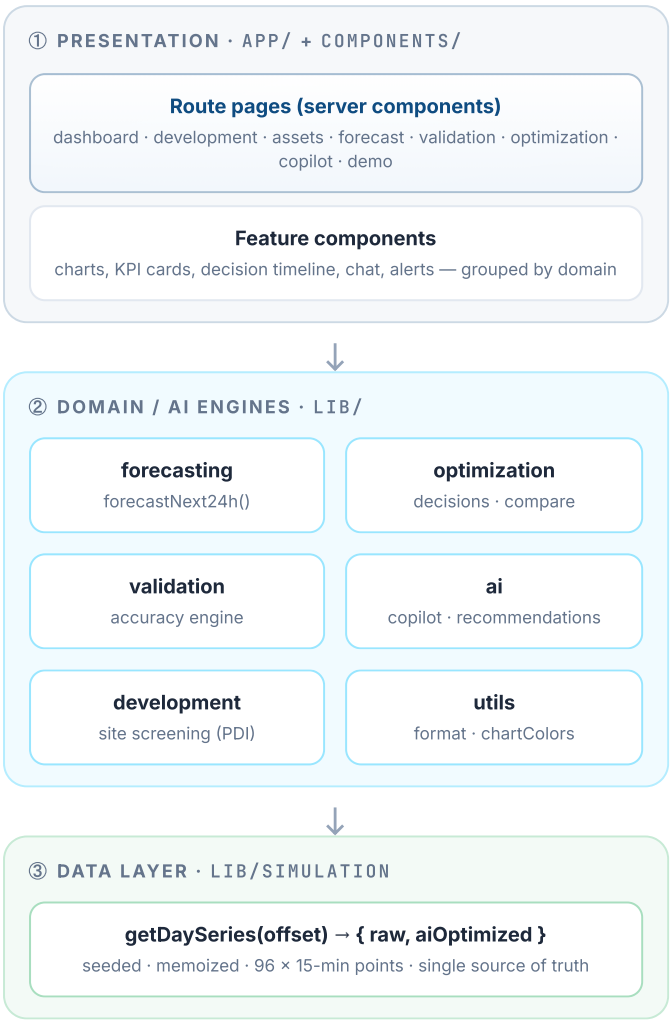
LAYER	TECHNOLOGY	RESPONSIBILITY IN WATTSYNC AI
Presentation	`app/` · `React` · `Recharts`	8 route pages rendered as server components; charts, KPI tiles, decision timeline, chat UI.
Domain / AI	`lib/forecasting` · `lib/optimization` · `lib/validation` · `lib/ai` · `lib/development`	Pure TypeScript engines. Deterministic, explainable, unit-testable — no framework coupling.
Data	`lib/simulation`	Seeded generator of raw + AI-optimized 24h series at 15-min resolution; memoized cache.
API	`app/api/copilot/chat/route.ts`	Single POST route handler (force-dynamic) for Copilot; everything else is static/server-rendered.
Delivery	`Vercel`	Zero-config import, `next build`, edge/serverless deploy.

Design principle — dependency minimalism. Seven runtime dependencies. No state manager, no ORM, no data-fetching library, no chart-config sprawl. Fewer moving parts → faster cold starts, smaller attack surface, trivially deployable.

Runs anywhere, breaks nowhere. With no database and no required keys, the app builds and serves entirely from computation. The Copilot degrades gracefully to a rule engine when no key is present — the demo cannot fail mid-presentation.

System Architecture

A three-tier separation inside one Next.js app: presentation (`app/`), domain engines (`lib/`), and the simulation data layer — with two clean provider seams for AI.



Architectural properties

- **Unidirectional data flow.** Presentation reads engines; engines read simulation. No upward dependencies, no circular imports.
- **Framework-free domain logic.** Every engine is pure TypeScript — no React, no Next — so it is testable and portable in isolation.
- **Server-first rendering.** Pages compute on the server as React Server Components; the browser ships mostly presentation.

Provider seams (the swap points)

`forecastNext24h()`

Returns the `Forecast24h` contract. Swapping the rule model for a trained ML model touches this one function — zero callers change.

`answerQuestion()`

Routes to Claude when a key is set, else the rule engine. The API route and UI never know which engine answered (beyond a labeled `mode`).

Server / client boundary

Server: all engines, simulation, the Claude provider (key never leaves the module), page rendering.

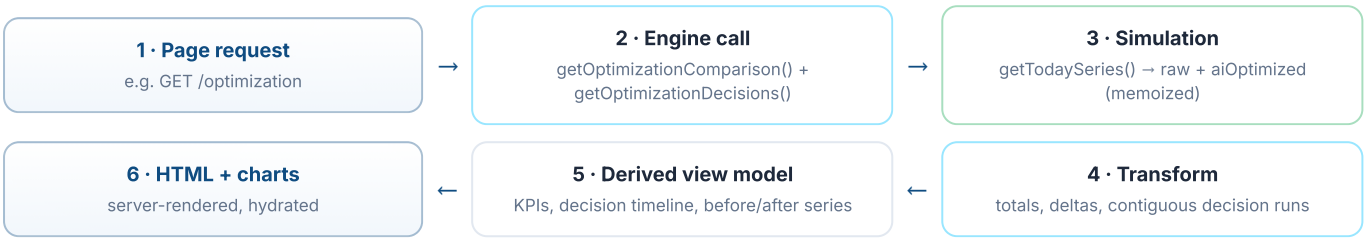
Client: interactive charts, the Copilot chat component, responsive navigation.

05 · RUNTIME

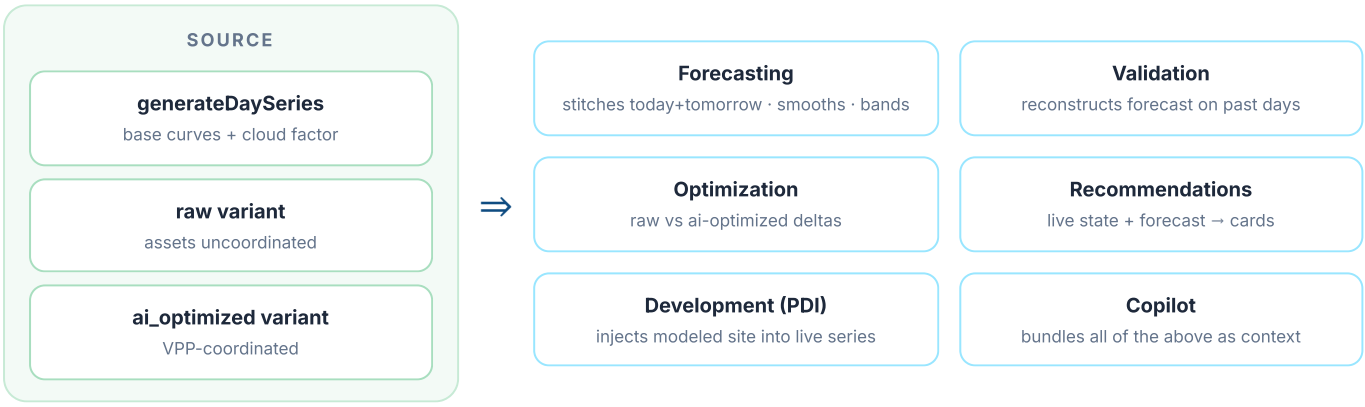
Data Flow

How one request becomes a fully-grounded page. The simulation runs once per calendar day (memoized); engines transform it; the page renders the result.

Request lifecycle



Simulation → engine fan-out



Determinism. Seed = `YYYYMMDD`. The same calendar day always produces identical curves, so forecasts, validations, and savings are stable and reproducible — essential for a live demo and for auditing claims.

Consistency by construction. Because forecasting, optimization, and the Copilot all read the *same* series, cross-page numbers never contradict. The Copilot cites the exact figures the dashboard shows.

Interval data point — the atomic record

FIELD GROUP	FIELDS (PER 15-MIN INTERVAL)
Generation	`solarKw · windKw · hydroKw · totalGenerationKw`
Demand	`buildingDemandKw · evDemandKw · totalDemandKw`
Storage & grid	`batterySocPercent · batteryFlowKw · gridImportKw · gridExportKw`
Context	`hour · ts · electricityPrice · weather`

06 · INTELLIGENCE

AI Architecture

Four intelligence layers. Each produces grounded numbers — real computation over the fleet, never canned text. Rule-based components are labeled honestly.

① Forecasting

Rule-based · seam ready for ML

Model: "persistence + physics" — next-24h stitched from today's remainder + tomorrow's seeded curve, 5-point moving-average smoothed.

- **Confidence bands** widen with horizon: ±5% now → ±20% at 24h.
- **Insights:** peak demand/generation, net surplus, surplus window, worst shortage, grid-risk rating, weather impact.
- **Grid risk** from on-peak (4–9pm) shortage depth: `>1200kW = high · >400kW = medium`.
- **Seam:** `forecastNext24h()` → `Forecast24h` contract.

② Validation

Answers "why trust the AI?"

Method: re-run the forecast for a completed day, compare to that day's realized (noisy) actuals. The gap is genuine error, not fabricated.

- **Accuracy** = weighted absolute error ÷ total actual energy (generation + demand).
- **Confidence calibration:** share of readings that landed inside the predicted band.
- **Largest miss** explained & attributed to solar vs wind, within operating hours (7am–7pm).
- **7-day trend** → improving / variable / steady.

③ Optimization

5-rule decision engine

Rules → explainable timeline. Each rule maps 1:1 to a VPP behavior; contiguous active runs become dated decisions with a magnitude and a numeric "why".

- Charge battery on renewable surplus
- Discharge battery to shave the evening peak
- Delay flexible EV load out of on-peak (resume off-peak)
- Export surplus when storage is full
- Import only when unavoidable (least-cost)

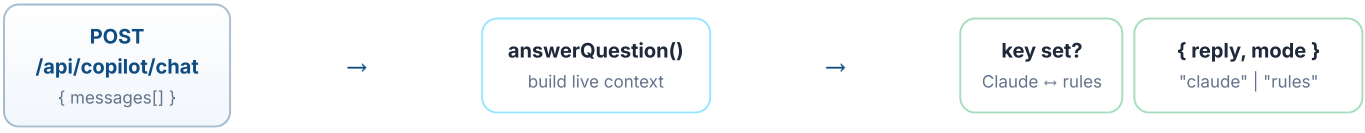
④ Copilot

Claude + rule fallback

Grounded operator Q&A. With `ANTHROPIC_API_KEY` set, questions go to `claude-opus-4-8` with the full live fleet snapshot as context; otherwise a deterministic intent router answers from the same data.

- **Server-only key** — never exposed to the browser.
- **Fail-safe:** 20s timeout / error → rule engine; the UI labels the engine used.
- **Grounding:** RM pricing, TOU windows, 0.4 kg CO_2/kWh baked into the system prompt.

Copilot request path



Recommendations engine (`lib/ai/recommendations.ts`) sits alongside these four: five time-aware rules emit fully-grounded cards — each with reason, expected impact, a 0–1 confidence score, a suggested action, and a validity window that expires once the peak passes.

07 · PLANNING → OPERATIONS

Project Development Intelligence (PDI)

The `/development` module screens candidate sites for the fleet's next asset, models benchmark economics, and — uniquely — injects the modeled site into today's *live* simulation to show the operational impact before a spade hits the ground.

Three-stage pipeline

① Site screening
Weighted multi-criteria score per candidate site → ranked, with a plain-English rationale.

→

② Feasibility economics
Capacity factor, annual/monthly yield, CAPEX, O&M, LCOE, payback, CO_2 avoided.

→

③ VPP integration impact
Inject the site's scaled solar curve into today's series → import avoided, extra export, renewable % lift.

Screening model — weighted criteria

CRITERION	WEIGHT	BASIS
Solar resource	`0.35`	Irradiance, normalized 4.5→6.5 kWh/m²/day
Interconnection	`0.25`	km to nearest substation (0→12 km)
Permitting	`0.20`	low / moderate / high risk band
Land cost & fit	`0.20`	RM/MW/yr vs lease benchmarks

Deterministic & explainable — every score traces to a stated assumption. A development-stage screening aid, not a bankable feasibility study.

Economic model — key benchmarks

PARAMETER	VALUE
Performance ratio	`0.79`
Tracking gain (greenfield)	`1.22×`
CAPEX — greenfield / rooftop	`$1.10 / $1.45 per W`
O&M	`$17 / kW-yr`
Capital recovery factor	`0.0858 (7%, 25 yr)`
Grid carbon	`0.4 t / MWh`

Candidate sites under screening

Greenfield

Kubang Pasu Solar Estate
Kedah · 6.0 MW · 5.8 kWh/m²/day · 2.1 km tie-in · moderate permitting

Rooftop

Shah Alam Logistics Rooftop
Selangor · 2.4 MW · behind-the-meter (NEM 3.0) · 0.4 km · low permitting

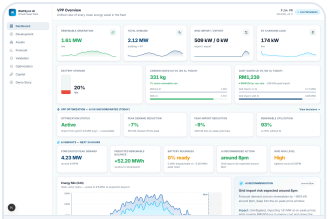
Greenfield

Bintulu Coastal Estate
Sarawak · 8.5 MW · 9.7 km tie-in · peat-edge EIA (DOE) · high permitting

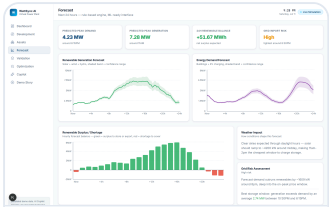
The planning→operations link is the differentiator. Rather than a static spreadsheet, PDI models the recommended site's hourly output against the live fleet — so a developer sees exactly how much evening grid import the new asset would displace *today*, and its renewable-share lift. A 5-stage development roadmap (site control → commissioning & VPP onboarding, months 0–24) frames the timeline.

Application Modules

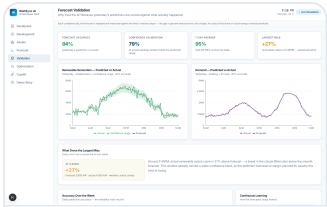
Eight routes, each backed by a domain engine. / redirects to /dashboard. Screenshots below are the live product.



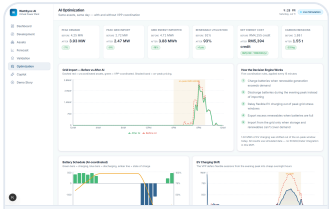
/dashboard — live KPIs, AI insights, optimization impact



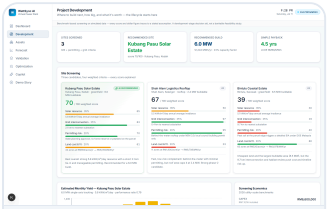
/forecast — 24h generation & demand, confidence bands



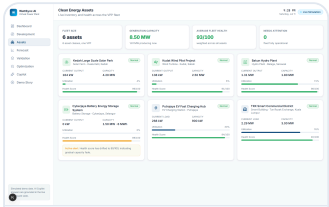
/validation — accuracy vs actuals, error explanation



/optimization — before/after, decision timeline

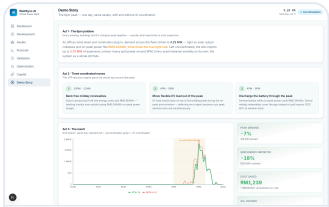


/development — site screening, feasibility, PDI



/assets — fleet inventory, utilization, health

/copilot — grounded operator Q&A



/demo — scripted 60-second walkthrough

MODULE	BACKING ENGINE	WHAT IT RENDERS
Dashboard	`simulation + recommendations + optimization`	Live KPIs, AI insight cards, energy mix & demand-vs-supply charts, battery SOC, alerts.
Development	`lib/development`	Ranked site scorecards, monthly yield, feasibility economics, VPP integration impact, roadmap.
Assets	`lib/simulation (deriveAssets)`	Per-asset capacity, live output, utilization, health score, alerts across the 6-asset fleet.
Forecast	`lib/forecasting`	Next-24h generation/demand with bands, surplus/shortage, weather impact, grid-risk rating.
Validation	`lib/validation`	Forecast-vs-actual chart, accuracy & confidence, named largest miss, 7-day reliability trend.
Optimization	`lib/optimization`	Before/after KPI grid, grid-import comparison, battery & EV schedules, AI decision timeline.
Copilot	`lib/ai/copilot + API route`	Chat UI with suggested prompts; each reply labeled Claude or rule-engine.
Demo	`composes all engines`	Three-act narrative: the 6pm problem → three coordinated moves → the measurable result.

09 · SERVER SIDE

Backend & Database Architecture

The MVP is intentionally **stateless**. There is no database; the simulation *is* the backend. One dynamic API route exists — the Copilot. This section is explicit about what is built vs. designed-for-later.

Current backend (implemented)

Compute

In-process engines

All logic runs server-side in the Next.js runtime. No external services, no queues, no workers.

API surface

One route handler

`POST /api/copilot/chat` (force-dynamic). Validates JSON, extracts the last user message, returns `{ reply, mode }`.

State

Memoized cache

Day series cached by seed in a `Map`, capped at the most recent few days so a long-lived process can't grow unbounded.

Data model — in-memory, typed

Core entities (TypeScript interfaces, no persistence)

- `Asset` — id, type, location, capacity, health, status
- `IntervalPoint` — one 15-min record (gen/demand/storage/grid/context)
- `DaySeries` — 96 points × { raw · aiOptimized }
- `Forecast24h · DayValidation · OptimizationComparison`
- `CandidateSite · FeasibilityReport · Recommendation · Alert`

Configuration & secrets

- `ANTHROPIC_API_KEY` — optional, server-side env var only
- No connection strings, no credentials, no user data
- `.env.example` documents the single optional var

> **No persistence today.** Nothing is written to disk or a database. Every value is recomputed per request from the deterministic seed.

Persistence layer — designed, not yet built

Future roadmap

PLANNED TABLE	PURPOSE
`fleets / assets`	Multi-tenant asset registry replacing the static demo fleet.
`time_series`	Real telemetry (SCADA / meter / inverter) — the `raw` variant becomes measured data.
`decision_audit_log`	Every optimization decision persisted for audit and post-hoc validation.
`copilot_history`	Persistent conversation memory for the Copilot.

Target: Supabase / Postgres. The schema is designed (per project README); no persistence code ships in the MVP.

10 · OPERATIONS

Deployment, Security & Scalability

Zero-config delivery on Vercel; a small, well-defined security surface; and a stateless design that scales horizontally by default.

Deployment topology

Git repository
import at vercel.com/new

→

Vercel build
next build · zero config

→

Edge / serverless
static pages + one dynamic route

Build

`npm run build` → optimized server components. No build-time secrets required.

Runtime env

Optional `ANTHROPIC_API_KEY` in Project → Environment Variables. Absent = rule-engine Copilot.

Local dev

`npm run dev` → localhost:3000, redirects to `/dashboard`.

Security posture

- **Server-only secret.** The API key is read only inside the Claude provider module and never imported by client code — never shipped to the browser.
- **Fail-strict AI.** Bad output shapes and refusals throw; the caller falls back rather than surfacing malformed data.
- **Minimal attack surface.** One POST endpoint; JSON validated; no file uploads, no auth tokens, no PII.
- **No sensitive data at rest.** Nothing persisted; no user accounts in the MVP.

Not yet in place: authentication / RBAC, rate limiting, and tenant isolation — required before real fleet data is connected. Tracked in the roadmap.

Scalability

- **Stateless by design.** No shared mutable state between requests → horizontal scaling is automatic on serverless.
- **Cheap compute.** Engines are O(96) over a day's intervals; a full page renders in milliseconds.
- **Memoization.** Day series computed once per seed and reused across all engines within a process.
- **Graceful AI degradation.** Copilot bounded by a 20s timeout + single retry, then instant rule fallback.

Scale path. Real fleets slot in behind the same engine seams; time-series volume moves to Postgres/Supabase; heavy forecasting can run as a separate model service without touching callers.

11 · HONEST SCOPE

Current Limitations vs Future Roadmap

What ships in the MVP today, and the defined path to production. The architecture's provider seams make each step a contained change.

CURRENT IMPLEMENTATION — MVP

- **Simulated fleet.** Data is a deterministic simulation, not live telemetry.
- **Rule-based forecasting.** Persistence+physics model, not a trained ML model.
- **Advisory optimization.** Decisions are computed & explained, but not dispatched to hardware.
- **Copilot without memory.** Grounded Q&A, but no persistent history and no action execution.
- **No persistence / multi-tenant.** Single in-memory fleet; nothing stored.
- **No auth.** Open demo; no accounts or RBAC.

FUTURE ROADMAP — TO PRODUCTION

- **1 · Real data ingestion.** SCADA / meter / inverter feeds replace the simulation seam; `raw` becomes telemetry.
- **2 · ML forecasting.** Trained model on weather + calendar features swaps into `forecastNext24h()` — one file.
- **3 · Copilot memory & actions.** Persistent history; close the loop from answer to dispatch action.
- **4 · Persistence & multi-tenant.** Supabase/Postgres for fleets, time series, decision audit logs.
- **5 · Dispatch integration.** Real battery/EVSE control via OpenADR / OCPP.

Roadmap sequence — each step is a contained swap

1 · Ingestion	2 · ML forecast	3 · Copilot actions	4 · Persistence	5 · Dispatch
simulation seam → real feeds	one-function swap	memory + dispatch loop	Postgres / multi-tenant	OpenADR / OCPP

Why the seams matter. Because the AI is isolated behind `forecastNext24h()` and `answerQuestion()`, and every page reads a single data contract, the MVP-to-production path is a series of localized replacements — not a rewrite. The demo's honesty (labeled rule engines, computed not canned figures) is exactly what makes the production story credible.

12 · VERIFY IT YOURSELF

Technical Evidence

Everything in this document is reproducible from the codebase. Links below marked TO ADD are not yet populated in the repository.

Source code

GitHub repository

`github.com/9Starz/wattsync-ai`

Full Next.js + TypeScript source, all engines, this document under `docs/`.

Live demo

Vercel deployment

`wattsync-ai.vercel.app`

Zero-config deploy. Open `/demo` for the 60-second story; ask `/copilot` a live question.

Walkthrough

Demo video

`‹ADD-VIDEO-LINK›`

Presenter script & recording checklist in `docs/DEMO.md`.

Reproduce locally

```
# install & run
npm install
npm run dev   # http://localhost:3000 → /dashboard
npm run build # production build (used by Vercel)

# optional Claude Copilot
cp .env.example .env.local && set ANTHROPIC_API_KEY
```

Where each claim lives in the code

CLAIM IN THIS DOCUMENT	SOURCE OF TRUTH
Deterministic simulation, 96×15-min, raw + AI variants	`lib/simulation/generateDaySeries.ts`
Forecast model & confidence bands	`lib/forecasting/forecastEngine.ts`
Accuracy validation against actuals	`lib/validation/engine.ts`
5-rule decision engine & before/after	`lib/optimization/decisions.ts · compare.ts`
Grounded Copilot + Claude provider	`lib/ai/copilot.ts · claudeProvider.ts`
PDI screening & feasibility	`lib/development/engine.ts · sites.ts`
Fleet (6 assets) & candidate sites	`lib/simulation/assets.ts · lib/development/sites.ts`
Design tokens (this doc's palette)	`docs/DESIGN_SYSTEM.md · app/globals.css`

Demo fleet. Kedah LSS Park (4.2 MW solar) · Kudat Wind Pilot (2.5 MW) · Bakun Hydro (1.8 MW) · Cyberjaya BESS (1.5 MW / 6 MWh) · Putrajaya EV Hub (0.9 MW) · TRX Smart District (3.0 MW). Real Malaysian sites; calculations are engine-computed, not scripted.